- **元素节点——插入**
  - jQuery 中封装了在指定位置动态插入元素节点的方法，其用法如下代码所示：

```
<script>
  // 待插入的元素节点
  let tr = $(`
    <tr>
      <td>学员</td>
      <td>17</td>
      <td>女</td>
      <td>13632369876</td>
      <td>
        <button class="btn btn-xs btn-info edit">编辑</button>
        <button class="btn btn-xs btn-danger delete">删除</button>
      </td>
    </tr>
  `);

  // 1. 参照父元素的位置插入
  $('tbody').append(tr);
  $('tbody').prepend(tr);

  // 2. 参照兄弟元素的位置插入
  $('tbody').eq(2).after(tr);
  $('tbody').eq(3).before(tr);
</script>
```

  - 总结：
    - `append`、`prepend` 以父元素为参考分别在结尾处和开头处插入新的元素节点
    - `after`、`before` 以当前元素为参考在之前或之后插入新的元素节点
    - `append`、`prepend`、`after`、`before` 均支持直接将 html 字符串做为节点插入

- **元素节点——删除**
  - jQuery 中封装了动态删除元素节点的方法，其用法如下代码所示：

```
<script>
  // 删除li元素
    $(this).parents('tr').remove();
```

```
</script>
```

- 总结：
  - `remove` 方法删除的是当前调用方法的元素节点
  - `empty` 方法删除的是当前调用方法的元素节点的后代

- **元素节点——复制**
  - jQuery 中封装了复制（克隆）元素节点的方法，其用法如下代码所示：

```
<script>
    // 通过复制获得新的节点
  $(this).parents('tr').clone(true);
</script>
```

- 总结：
  - `clone` 方法复制得到的元素节点仍是 jQuery 对象
  - 待复制的节点中如果有事件监听，需要为 `clone` 方法传入参数 `true`

- **元素节点——事件委托**
  - jQuery 中封装了事件委托的支持，其用法如下代码所示：

```
<script>
  // on 方法内置支持事件委托
    $('table').on('click', '.delete', function () {
    $(this).parents('tr').remove();
  })

  // 或者使用
  $('table').delegate('.delete', 'click', function () {
    $(this).parents('tr').remove();
  })
</script>
```

- 总结：
  - 事件委托需要为某个在 DOM 中已经存在的祖先元素添加事件监听
  - `delegate` 方法是 jQuery 中专门的事件委托的方法
  - `on` 方法中也内置支持事件委托，推荐使用 `on` 方法

- **位置和尺寸——位置**
  - jQuery 对获取元素位置进行了封装，使得在不同场景中获取元素位置十分方便，其用法如下代码所示：

```
<script>
  // 获取参照 html 文档的位置
  $('.box').offset();

  // 获取参照最近的已定位祖先元素位置
  $('.box').position();

  // 获取子元素滚动的距离
  $('.outer').scrollTop();
  $('.outer').scrollLeft();
</script>
```

  - 总结：
    - `offset` 方法获取元素参照 html 文档的位置，无论该元素是否采用了定位
    - `position` 方法获取元素参照最近的已定位的祖先元素的位置
    - `scrollTop/scrollLeft` 方法获取子元素滚动的位置（距离）
    - 了解一个细节：`offset` 计算位置时会忽略外边距（margin），而 `position` 计算位置时则以外边距（margin）为边界

- **位置和尺寸——尺寸**
  - jQuery 对获取元素尺寸进行了封装，使得在不同场景中获取元素尺寸十分方便，其用法如下代码所示：

```
<script>
    // 只包含内容区域尺寸大小
  $('.box').width();
  $('.box').height();

  // 包括内容区域 + 内边距尺寸大小
  $('.box').innerWidth();
  $('.box').innerHeight();

  // 包括内容区域 + 内边距 + 边框尺寸大小
  $('.box').outerWidth();
  $('.box').outerHeight();
```

```
</script>
```

- 总结：
  - `width/height` 方法获取元素尺寸大小时只包括盒子模型中的内容区域
  - `innerWidth/innerHeight` 方法获取元素尺寸大小时包括盒子模型中的内容区域 + 内边距
  - `outerWidth/outerHeight` 方法获取元素尺寸大小时包括盒子模型中的内容区域 + 内边距 + 边框
  - 注：`outerWidth/outerHeight` 方法传入参数值 `true` 获取元素尺寸大小包括内容区域 + 内边框 + 边框 + 外边距

- **动画——基础动画**
  - jQuery 中封装了元素显示/隐藏的快捷操作并且支持动画形式的交互效果，主要有以下几种用法：
    - 显示/隐藏

```
<script>
  // 设置盒子显示
  $('.box').show();
    // 设置盒子隐藏
  $('.box').hide();
    // 设置盒子显示/隐藏
  $('.box').toggle();
</script>
```

- 总结：
  - `show` 方法设置元素显示，实质是设置元素样式 `display: block`
  - `hide` 方法设置元素隐藏，实质是设置元素样式 `display: none`
  - `toggle` 方法交替设置元素显示/隐藏
  - `show`、`hide`、`toggle` 方法均可以接收时间（毫秒）做为参数，此时将产生动画效果

    - 淡入/淡出

```
<script>
  // 设置盒子显示
  $('.box').fadeIn();
    // 设置盒子隐藏
  $('.box').fadeOut();
    // 设置盒子显示/隐藏
  $('.box').fadeToggle();
```

```
</script>
```

- 总结：
  - `fadeIn` 方法设置元素显示，实质是设置元素样式 `opacity: 1; display: block;`
  - `fadeOut` 方法设置元素隐藏，实质是设置元素样式 `opacity: 0; display: none`
  - `fadeToggle` 方法交替设置元素显示/隐藏
  - fadeTo：淡出到某个程度：必写参数透明度的值（0-1）
  - `fadeIn`、`fadeOut`、`fadeToggle` 方法默认支持动画效果，接收时间（毫秒）做为参数时能够控制动画执行的速度
- 展开/折叠：滑动动画

```
<script>
    // 滑动效果
  // 设置盒子显示
  $('.box').slideDown();
    // 设置盒子隐藏
  $('.box').slideUp();
    // 设置盒子显示/隐藏
  $('.box').slideToggle();
</script>
```

- 总结：
  - `slideUp` 方法设置元素隐藏，实质上设置元素的宽高和内外边距以及 `overflow: hidden`
  - `slideDown` 方法设置元素显示，实质上设置元素的宽高和内外边距
  - `slideToggle` 方法交替设置元素的显示/隐藏
  - `slideUp`、`slideDown`、`slideToggle` 方法默认支持动画效果，接收时间（毫秒）做为参数时能够控制动画执行的速度

- **动画——自定义**
  - jQuery 中提供的基础动画主要是针对元素的显示/隐藏展开的，不仅如此 jQuery 还提供了 `animate` 方法支持开发者自定义更为丰富的动画效果，其用法如下代码所示：

```
<script>
  // 自定义动画
  $('.box').animate({
    marginLeft: 200,
    width: 300,
    height: 200,
```

```
    backgroundColor: 'red'
  }, 2000)
</script>
```

- 总结：
  - `animate` 方法支持开发者自定义 CSS 动画样式，并控制动画执行的速度
  - `animate` 只支持值为数值的 CSS 样式，默认以 `px` 为长度单位

- **动画——其它**
  - 延时设置
    - jQuery 不仅可以设置动画执行的速度，还能在动画执行前设置一定的延时，其用法代下代码所示：

```
<script>
  // 等待 1500 毫秒后再隐藏
    $('.box').delay(1500).hide();
  // 选改变盒子宽度为 400px 等待 1000 毫秒后再改变盒子的高度为 200px
  $('.box').animate({width: 400}, 500).delay(15000).animate({height: 200}, 500);
</script>
```

      - 总结：
        - `delay` 方法常用来设置动画的延时执行，接受时间（毫秒）做为参数
  - 终止动画

```
<script>
  // 只传一个 true 时，为 暂停
    $('.box').stop(true);
  // 传入两个 true 时，为 结束
    $('.box').stop(true, true);
</script>
```

    - 总结：
      - `stop` 只传一个 true 时，为暂停
      - `stop` 传入两个 true 时，为结束
  - 回调函数
    - 所有的 jQuery 动画方法都支持传入回调函数，该函数会在动画执行结束时立即执行，其用法如下代码所示：

```
<script>
  $('.box').fadeOut(500, function () {
    // 回调函数会在动画执行结束时被调用
    // 引入的 this 指向了执行动画的元素节点
    $(this).remove();
  })
</script>
```

    - 总结：
        - 回调函数在动画执行结束时被执行，回调函数中的 `this` 指向执行动画的元素节点x

- **hover事件**
    - 说明：由mouseenter和mouseleave封装起来的事件。
    - 写两个参数函数的时候，这两个函数会在鼠标移入和移出的时候先后执行。

```
$('div').hover(function () {
    console.log('111');
}, function () {
    console.log('222');
});
```

    - 写一个参数函数的时候，这个函数会在鼠标移入和移出的时候都执行。

```
$('div').hover(function () {
    console.log('hover');
})
```